Assignment 2 – Utility Class with Testing and Metrics

Name: Beyoncé del Mar Ramón Yánez

D.N.I: 02736665X

For this assignment, I decided to implement a utility class called StringUtils. The idea behind this class is to group together several useful operations related to string manipulation, which are common in many software applications.

The class includes five methods: checking if a string is a palindrome, counting character occurrences, normalizing whitespace, validating email formats, and reversing the order of words in a sentence.

I chose this theme because it allows for different types of logic, including iteration, conditionals, and input validation. It also makes it easier to apply testing techniques such as equivalence class partitioning and boundary value analysis.

The class contains five methods, each designed to solve a specific problem:

- isPalindrome(String s): checks whether a given string reads the same forwards and backwards. It ignores spaces and is case-insensitive.
- countOccurrences(String s, char c): counts how many times a given character appears in a string.
- normalizeWhitespace(String s): removes extra spaces from a string, ensuring that only single spaces remain between words.
- isValidEmail(String s, PatternProvider provider): validates whether a string matches a valid email format using a regex provided by an external interface.
- reverseWords(String s): reverses the order of words in a sentence without modifying the words themselves.

Each method was implemented with simplicity and clarity in mind, while still handling edge cases where necessary.

A custom exception called EmptyStringException was created to handle invalid inputs.

This exception is thrown when the input string is either null or empty in methods where a valid string is required, such as isPalindrome.

The purpose of defining a custom exception instead of using a generic one is to make the error more specific and easier to understand during debugging and testing.

UNIT TEST QUALITY

ECP (Equivalence Class Partitioning)

For the method isPalindrome, I identified several equivalence classes:

- Valid input where the string is a palindrome (e.g., "racecar")
- Valid input where the string is not a palindrome (e.g., "hello")
- Invalid input such as an empty string
- Invalid input such as null

These classes ensure that all possible types of input are covered without testing every possible string.

Input Type	**Example**	**Expected Result**
Valid palindrome	"racecar"	true
Valid non-palindrome	"hello"	false
Empty string	""	Exception
Null input	null	Exception

BVA (Boundary Value Analysis)

Boundary value analysis was applied based on the length of the input string:

- Strings of length 1 (e.g., "a"), which should always be considered palindromes
- Strings of length 2, both equal (e.g., "aa") and different (e.g., "ab")

These cases help verify that the method behaves correctly at the limits of valid input sizes.

Case	Input	Expected
Minimum length	"a"	true
Boundary equal	"aa"	true
Boundary different	"ab"	false

EXCEPTION TESTING

Exception handling was tested by providing invalid inputs such as empty strings.

For example, when calling isPalindrome with an empty string, the method correctly throws an EmptyStringException.

This confirms that the validation logic works as expected and prevents incorrect usage of the methods.

TEST EXECUTION

The unit tests were executed using JUnit.
All tests passed successfully, confirming the correctness of the implementation.

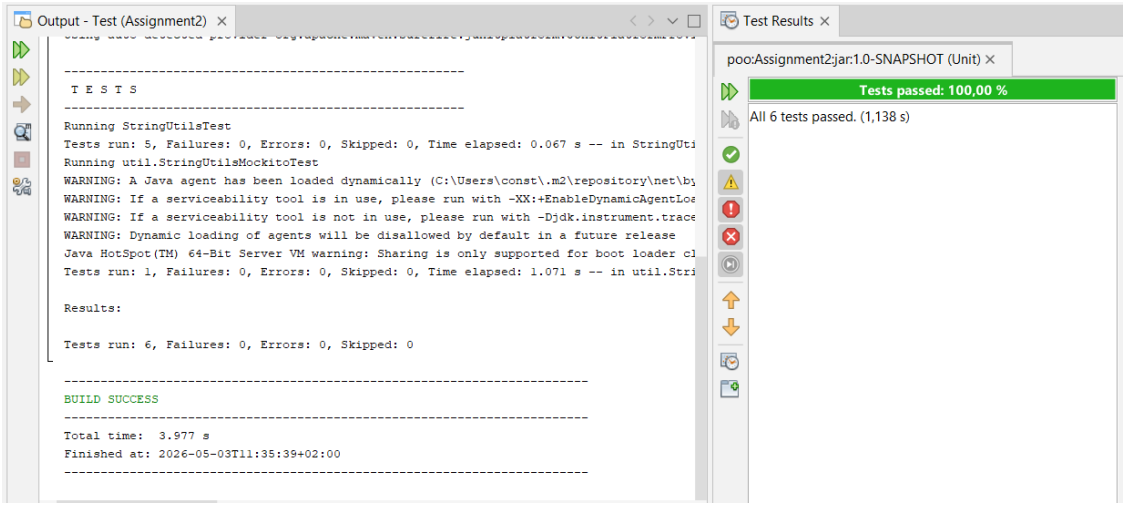


Figure 1: Test execution results showing all tests passed successfully.

This result shows that all implemented tests executed correctly without failures.

PROJECT STRUCTURE

The project follows the standard Maven structure, separating source code and test code into different packages.

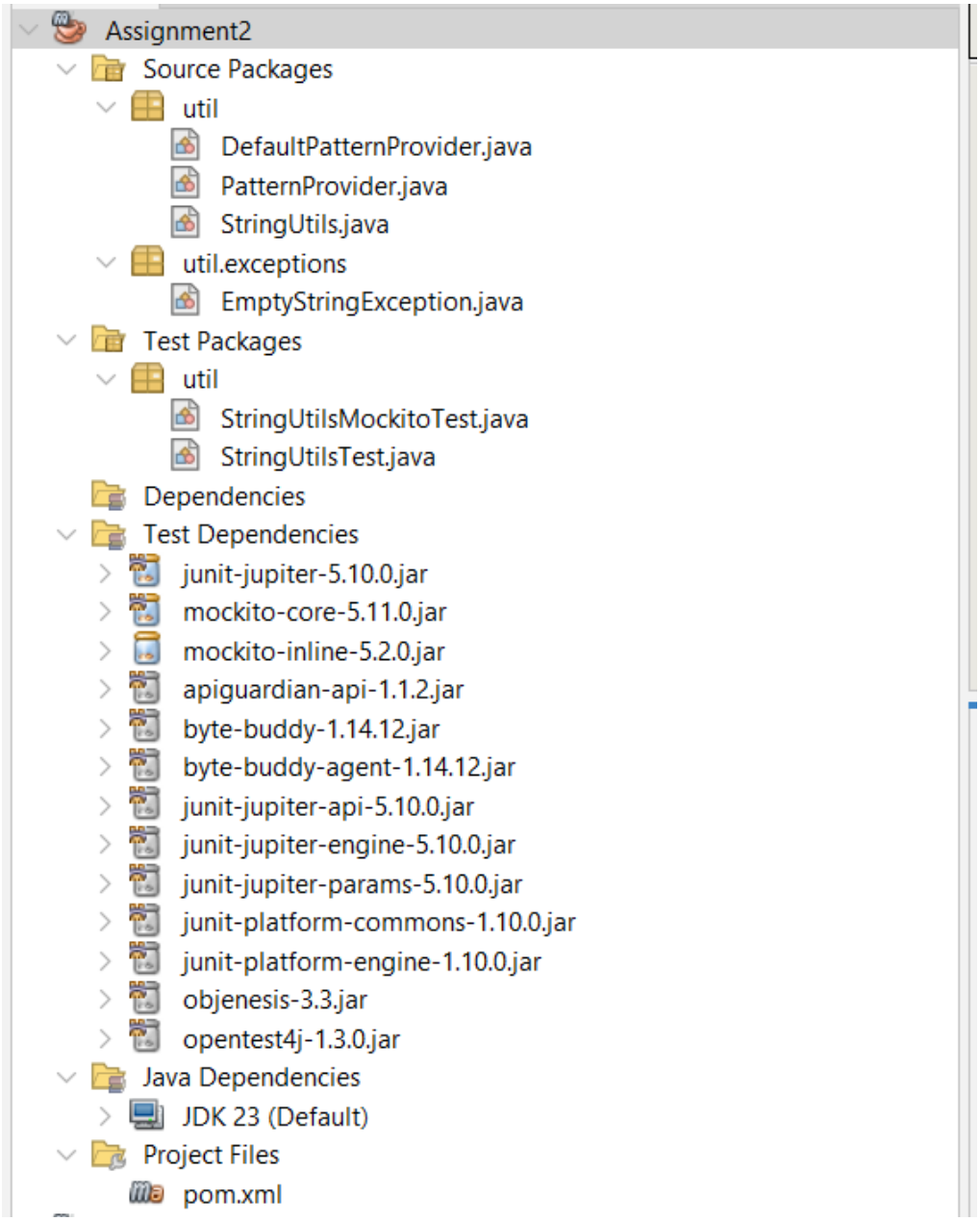


Figure 2: Project structure showing source and test packages.

This structure follows the standard Maven convention, improving project organization and maintainability.

MOCKITO TEST

Mockito was used to mock the PatternProvider interface.

This allows the isValidEmail method to be tested independently from the actual implementation of the pattern provider.

The mock was configured to return a predefined regex pattern, and its interaction was verified during the test execution.

```java
package util;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
public class StringUtilsMockitoTest {

    @Mock
    PatternProvider mockProvider;

    @Test
    void testValidEmailWithMock() {
        // Definir comportamiento del mock
        when(mockProvider.getEmailRegex())
                .thenReturn("^[A-Za-z0-9+_.-]+@(.+)$");

        // Ejecutar método
        boolean result = StringUtils.isValidEmail("user@test.com", mockProvider);

        // Verificar resultado
        assertTrue(result);

        // Verificar que se llamó al mock
        verify(mockProvider).getEmailRegex();
    }
}
```

Figure 3: Mockito test implementation using a mocked PatternProvider.

CYCLOMATIC COMPLEXITY

The cyclomatic complexity was calculated for three methods, including isPalindrome.

In this method, there are multiple decision points: one for input validation, one loop, and a conditional inside the loop. This results in a complexity of 4.

This value indicates that the method has moderate complexity but is still easy to understand and maintain.

Refactoring was considered but not necessary, as the method is already clear and well-structured.

Cyclomatic complexity was also calculated for other methods.

countOccurrences:

Cyclomatic Complexity: 2

- One loop

- No complex branching

reverseWords:

Cyclomatic Complexity: 2

- One loop

- Simple condition inside loop

Overall, the complexity values are low to moderate, indicating that the methods are easy to understand and maintain. Refactoring was not necessary.

CK METRICS

CK metrics were calculated to evaluate the quality of the class design.

WMC (Weighted Methods per Class) is 5, indicating a moderate level of complexity.

CBO (Coupling Between Objects) is 2, showing low coupling between components.

RFC (Response for Class) is 6, indicating a moderate number of possible method responses.

Overall, the metrics indicate that the class is well-structured and maintainable.

CK Metrics Results

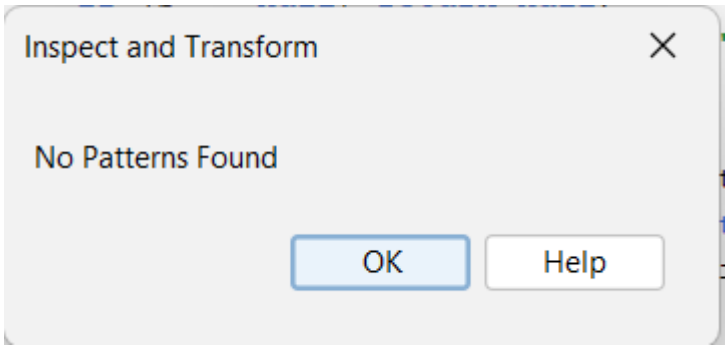
Metric	Value	Interpretation
WMC	5	Moderate complexity
CBO	2	Low coupling
RFC	6	Moderate response

```java
public class StringUtils {
    public static boolean isPalindrome(String s) {
        if (s == null || s.isEmpty()) {
            throw new EmptyStringException("Input cannot be empty");
        }

        String clean = s.replaceAll("\\s+", "").toLowerCase();

        int left = 0;
        int right = clean.length() - 1;

        while (left < right) {
            if (clean.charAt(left) != clean.charAt(right)) {
                return false;
            }
            left++;
            right--;
        }

        return true;
    }

    public static int countOccurrences(String s, char c) {
        if (s == null) return 0;

        int count = 0;
        for (char ch : s.toCharArray()) {
            if (ch == c) count++;
        }
        return count;
    }

    public static String normalizeWhitespace(String s) {
        if (s == null) return null;
        return s.trim().replaceAll("\\s+", " ");
    }

    public static boolean isValidEmail(String s, PatternProvider provider) {
        if (s == null || s.isEmpty()) return false;
        return s.matches(provider.getEmailRegex());
    }

    public static String reverseWords(String s) {
        if (s == null) return null;

        String[] words = s.split(" ");
        StringBuilder result = new StringBuilder();

        for (int i = words.length - 1; i >= 0; i--) {
            result.append(words[i]);
            if (i > 0) result.append(" ");
        }

        return result.toString();
    }
}
```

STATIC ANALYSIS

Static analysis was performed using the NetBeans "Inspect and Transform" tool with the default configuration over the entire project.

The analysis did not detect any issues or warnings ("No Patterns Found"), indicating that the code is clean and follows good programming practices.



OPTIONAL EXTENSION

Code coverage analysis was not included due to compatibility issues between JaCoCo and the Java version used.

However, the test suite covers both normal and edge cases.

CONCLUSION

In this assignment, I implemented a utility class with several string-related methods and applied different software testing techniques.

Unit testing, equivalence class partitioning, boundary value analysis, and mocking were all used to ensure the correctness of the implementation.

Additionally, code quality was evaluated using cyclomatic complexity and CK metrics.

Overall, the project demonstrates a structured approach to software development, combining implementation, testing, and analysis.